PROGRAMMING WITH PYTHON
SUBJECT CODE : 202044504

DEPARTMENT OF INFORMATION TECHNOLOGY
SEMESTER 5

NAME : Rana Riyancee
ENROLLMENT NO. : 12402080601119
BATCH :IT_B (2B8)
ACADEMIC YEAR : 2026-27

Github Link :
https://github.com/riyanceerana-art/12402080601119_Python_Assignment/tree/main/Assignment1

ASSIGNMENT-1

PROBLEM STATEMENT 1: You are given records of students in the form enrollment, name, semester, CPI and a list of subject marks. Build a program that stores the records using lists, tuples and dictionaries. For each semester, print the top K students by CPI. If CPI is the same, prefer the student with higher average marks; if still tied, prefer lexicographically smaller enrollment number. Also print the subject-wise topper for every subject code.

CODE :
```
def main(): n, k, m = map(int, input().split()) if not (1 <= n <=
100000): raise ValueError("Number of students must be between 1 and
100000.")


    if not (1 <= k <= 50):
        raise ValueError("K must be between 1 and 50.")


    if not (1 <= m <= 12):
        raise ValueError("Number of subjects must be between 1 and 12.")
    semester_students = {}
    subject_toppers = {} for i in range(1,
    m + 1): subject_toppers[f"S{i}"] =
    (-1, [])

    for _ in range(n):
        data = input().split()

        enrollment = data[0] name =
        data[1] semester = int(data[2])
        cpi = float(data[3]) marks =
        list(map(int, data[4:])) if
        len(marks) != m:


            raise ValueError( f"{enrollment} must have exactly {m}
                subject marks."
            )

        if any(mark < 0 or mark > 100 for mark in marks):
            raise ValueError("Marks must be between 0 and 100.")
        if not (1 <= semester <= 8):
            raise ValueError("Semester must be between 1 and 8.")
```



```
    if not (0.0 <= cpi <= 10.0): raise ValueError("CPI must be
    between 0.0 and 10.0.") average_marks = sum(marks) / m

    student = (
        enrollment,
        name,
        cpi,
        average_marks,
        marks
    )

    if semester not in semester_students:
        semester_students[semester] = []
    semester_students[semester].append(student


    ) for i, mark in enumerate(marks, start=1):


        subject = f"S{i}" highest_mark, toppers =
        subject_toppers[subject] if mark > highest_mark:
        subject_toppers[subject] = (mark, [enrollment])

        elif mark == highest_mark:
            toppers.append(enrollment)
for semester in
    sorted(semester_students): students =
    semester_students[semester]
    students.sort( key=lambda student: (
            -student[2],
            -student[3],
            student[0]
        )
    )
    top_students = students[:k]

    enrollments = [student[0] for student in top_students] print(f"Semester

     {semester}:", *enrollments)
```

```
for i in range(1, m + 1): subject = f"S{i}"
        highest_mark, toppers = subject_toppers[subject]
        toppers.sort()
        print(f"{subject}:",
*toppers) if name == " main ": main()
```

OUTPUT:

```
5 2 3
2201 Asha 5 8.90 90 82 75
2202 Bharat 5 8.90 88 95 70
2203 Chaitra 3 9.10 91 88 92
2204 Dev 5 7.80 76 80 85
2205 Esha 3 9.10 91 90 88
Semester 3: 2203 2205
Semester 5: 2202 2201
S1: 2203 2205
S2: 2202
S3: 2203
```

PROBLEM STATEMENT 2: Given n candidate passwords, classify each as STRONG, WEAK_LENGTH, WEAK_PATTERN or COMPROMISED. A strong password must contain at least one lowercase letter, uppercase letter, digit and one special symbol from $#@. Its length must be between 6 and 12. It must not contain any dictionary word from a banned list as a contiguous substring, ignoring case. It must also not contain the same character more than 3 times consecutively.

CODE :

```
import re class
TrieNode:
    def init (self):
        self.children = {} self.is_end
        = False

class Trie: def _
    init (self):
```

```
        self.root = TrieNode()
    def insert(self, word): node
        = self.root for char in
        word.lower():
            if char not in node.children:
```

Riyancee Rana (12402080601119)



```
                node.children[char] = TrieNode()

    node = node.children[char] node.is_end =

    True def contains_banned_word(self,

    password):

        password = password.lower() for

        start in range(len(password)):

            node = self.root

            for end in range(start, len(password)):
                char = password[end] if
                char not in node.children:
                break

                node = node.children[char] if
                node.is_end:
                    return Truereturn False

def
    has_required_characters(password):
    has_lower = False has_upper = False
    has_digit = False has_special = False

    for char in password:
        if char.islower():
            has_lower = True
        elif char.isupper():
        has_upper = True
        elif char.isdigit():
            has_digit = True
        elif char in "$#@":
```

```
        has_special = True

    return has_lower and has_upper and has_digit and has_special def

has_repeated_character(password):

    if not password:
```

Riyancee Rana (12402080601119)

```
return False

    consecutive_count = 1

    for i in range(1, len(password)):
        if password[i] == password[i - 1]: consecutive_count
            += 1

            if consecutive_count > 3: return
                True
        else:
            consecutive_count = 1

    return False
def classify_password(password, banned_trie):
    if len(password) < 6 or len(password) > 12: return
        "WEAK_LENGTH"

    if not has_required_characters(password): return
        "WEAK_PATTERN"
    if has_repeated_character(password): return
        "WEAK_PATTERN"

    if banned_trie.contains_banned_word(password):
        return "COMPROMISED"
return
"STRONG" def main(): b =
int(input().strip()) if not (1
<= b <= 10000):
        raise ValueError("Number of banned words must be between 1 and 10000.")
    banned_trie = Trie()

    for _ in range(b):
        word = input().strip() if
```

```
        word:
        banned_trie.insert(word)
    n = int(input().strip())

    if not (1 <= n <= 100000):
        raise ValueError("Number of passwords must be between 1 and 100000.")

    for index in range(1, n + 1):
```

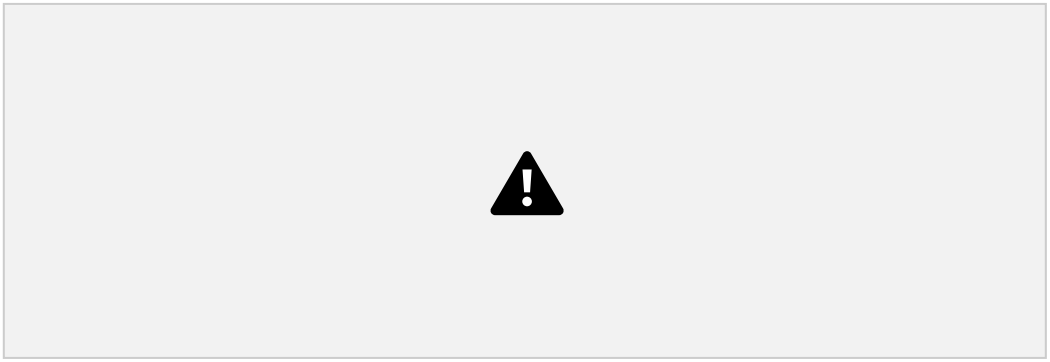
Riyancee Rana (12402080601119)




```
password =
        input().rstrip("\n")
        result = classify_password(password, banned_trie) print(f"{index}:
         {result}")

if name == " main ": main()
```

OUTPUT:



PROBLEM STATEMENT 3: Implement a recursive expression evaluator for integer expressions containing non-negative integers, +, -, *, parentheses and variable references. Each variable may be defined by another expression. Detect cyclic variable dependencies and report CYCLE. Use memoization so that repeated variable evaluations are not recomputed.

CODE :

```
import re class
ExpressionEngine:
    def init (self, variables): self.variables
        = variables
        self.memo = {}
        self.visiting = set() self.variable_pattern = re.compile(r"^[A-Za
    z_][A-Za-z0-9_]*$") def tokenize(self, expression):

        tokens = [] i
        = 0
```

```
        while i < len(expression):

            if
                expression[i].isspace()
                : i += 1 continue
    if expression[i].isdigit():
                j = i
```

Riyancee Rana (12402080601119)


```
while j <
                len(expression) and expression[j].isdigit():
                    j += 1

                tokens.append(("NUMBER", expression[i:j])) i
                = j
                continue


            if expression[i].isalpha() or expression[i] == "_": j
                = i

                while ( j <
                    len(expression)
                    and (expression[j].isalnum() or expression[j] == "_")
                ):
                    j += 1
                tokens.append(("VARIABLE", expression[i:j]))i = j continue

            if expression[i] in "+-*()":
                tokens.append((expression[i],
                expression[i])) i += 1 continue

            raise ValueError("Invalid character in expression.") return

        tokens

    def evaluate_variable(self, name):

        if name in self.memo:
            return self.memo[name]

        if name not in self.variables:
```

```
            raise ValueError("Unknown variable.")

        if name in self.visiting:
            raise RuntimeError("CYCLE")

        self.visiting.add(name)

        try:
            tokens = self.tokenize(self.variables[name])
            value, position = self.parse_expression(tokens, 0)
            if position != len(tokens):
```

Riyancee Rana (12402080601119)



```
raise ValueError("Invalid expression.")
            self.memo[name] = value

            return value

        finally:
            self.visiting.remove(name)

    def parse_expression(self, tokens, position):

        value, position = self.parse_term(tokens, position)
        while position < len(tokens):
            operator = tokens[position][0]
            if operator not in ("+", "-"):
                break
            position += 1

            right_value, position = self.parse_term(tokens, position)

            if operator == "+":
                value += right_value
            else:
                value -= right_value

        return value, position

    def parse_term(self, tokens, position):
```

```
    value, position = self.parse_factor(tokens, position) while

    position < len(tokens):

        operator = tokens[position][0]

        if operator != "*":
            break

        position += 1
```

Riyancee Rana (12402080601119)


```
        right_value, position = self.parse_factor(tokens, position) value

        *= right_value

    return value, position
def parse_factor(self, tokens, position):
    if position >= len(tokens): raise

    ValueError("Invalid expression.")

    token_type, token_value = tokens[position]

    if token_type == "NUMBER":

        return int(token_value), position + 1 if

    token_type == "VARIABLE":

        if token_value not in self.variables: raise

        ValueError("Unknown variable.") value =

        self.evaluate_variable(token_value) return

        value, position + 1

    if token_type == "(":

        value, position = self.parse_expression(
            tokens, position
```

```
            + 1
        )

        if (
            position >= len(tokens) or
            tokens[position][0] != ")"
        ):
            raise ValueError("Missing closing parenthesis.")
        return value, position + 1
    raise ValueError("Invalid expression.")
def evaluate(self, expression):
    tokens = self.tokenize(expression) if not
    tokens: raise ValueError("Empty
    expression.")
```

Riyancee Rana (12402080601119)

 value,

```
    position = self.parse_expression(tokens, 0)

    if position != len(tokens): raise
        ValueError("Invalid expression.")
    return value

def main():
    v = int(input().strip())

    if v < 1:
        raise ValueError("Number of variables must be at least 1.")
    variables = {} for
    _ in range(v): line
    = input().strip() if
    "=" not in line:
    print("INVALID
    ") return

        name, expression = line.split("=", 1) name =
        name.strip() expression = expression.strip() if not
        re.match(r"^[A-Za-z_][A-Za-z0-9_]*$", name):
        print("INVALID") return
        if not expression:
            print("INVALID")
            return
```

```
        variables[name] = expression
    final_expression = input().strip()

    try:
        engine = ExpressionEngine(variables) result
        =
        engine.evaluate(final_expression)
        print(result)

    except RuntimeError as error:
        if str(error) == "CYCLE":
            print("CYCLE")
        else:
            print("INVALID")
    except (ValueError, RecursionError):
        print("INVALID")
```

Riyancee Rana (12402080601119)

if name == " main ": main()

OUTPUT:



PROBLEM STATEMENT 4: A CSV file contains transaction_id, account_id, type, amount and timestamp. Create a program that reads the file, validates every row, writes valid credit and debit transactions into separate CSV files, writes rejected rows into error.csv with reason, and finally prints account-wise balance changes. The program must continue processing after invalid rows using exception handling.

CODE :

```
import csv
from datetime import datetime

def validate_transaction(row): required_fields = ["tid",
    "acc", "type", "amount", "time"]

    for field in required_fields:
        if field not in row or not row[field].strip(): return
```

```
            False, f"Missing {field}"

    transaction_type = row["type"].strip().upper()

    if transaction_type not in ("CREDIT", "DEBIT"):
        return False, "Invalid transaction type"

    try:
        amount = float(row["amount"])
    except ValueError: return False,
        "Amount is not numeric"

    if amount <= 0: return False, "Amount must be
        greater than zero"

    try:
        datetime.strptime(
            row["time"].strip(),
```

Riyancee Rana (12402080601119)




```
            "%Y-%m-%dT%H:%M:%S"
        )
    except ValueError: return False,
    "Invalid timestamp" return True, ""

def

    process_file(filename):
        balances = {}try:

        with open(filename, "r", newline="", encoding="utf-8") as infile:

            reader = csv.DictReader(infile)

            required = {"tid", "acc", "type", "amount", "time"}

            if not reader.fieldnames or not required.issubset( set(reader.fieldnames)
            ):
                print("Invalid CSV header.")
                return
```

```
with open(
    "credit.csv", "w", newline="", encoding="utf-8"
) as credit_file, \ open(
    "debit.csv", "w", newline="", encoding="utf-8"
) as debit_file, \ open(
    "error.csv", "w", newline="", encoding="utf-8"
) as error_file:

    credit_writer = csv.DictWriter( credit_file,
        fieldnames=reader.fieldnames
    )

    debit_writer = csv.DictWriter( debit_file,
        fieldnames=reader.fieldnames
    )
    error_fields = reader.fieldnames + ["reason"] error_writer

    = csv.DictWriter(
```

Riyancee Rana (12402080601119)



error_file,

```
        fieldnames=error_fields
    )
    credit_writer.writeheader()
    debit_writer.writeheader()
    error_writer.writeheader()f or row in reader:
    try: valid, reason = validate_transaction(row)

        if not valid:
            row_with_error = row.copy()
            row_with_error["reason"] = reason
            error_writer.writerow(row_with_error)
            continue

        account = row["acc"].strip() transaction_type
        = row["type"].strip().upper() amount =
        float(row["amount"])
balances.setdefault(account, 0)

        if transaction_type == "CREDIT":
```

```
                credit_writer.writerow(row)
                balances[account] += amount

            else:
                debit_writer.writerow(row)
                balances[account] -= amount

        except Exception as error:
            row_with_error = row.copy()
            row_with_error["reason"] = str(error)
            error_writer.writerow(row_with_error
            )

    except FileNotFoundError:
        print("Input file not found.")
        return
    except Exception as error: print("Error:",
        error) return
    sorted_balances = sorted(
        balances.items(), key=lambda
        item: abs(item[1]),
        reverse=True
```

Riyancee Rana (12402080601119)



```
    print("Account-wise balance changes:") for
    account, balance in sorted_balances:
        if balance.is_integer():
            print(account, int(balance))
        else: print(account,
            balance)


    print("Files created: credit.csv, debit.csv, error.csv") def

main():

    filename = input("Enter CSV file path: ").strip()

    if not filename: print("File path
    cannot be empty.") return
    process_file(filename)

if name == " main ": main()
```

OUTPUT:



PROBLEM STATEMENT 5: Design an OOP-based settlement engine with Account, Transaction and Bank classes. The engine must support deposit, withdraw and transfer operations, maintain transaction history, prevent overdraft, and rollback an entire batch if any transaction in the batch fails. Use custom exceptions and suitable getters/setters or properties. **CODE :** class BankError(Exception): pass
class
AccountNotFoundError(BankError):
pass
class InvalidAmountError(BankError):
pass
class InsufficientBalanceError(BankError):
pass
class Account:


def init
(self, account_id, balance=0):
self.account_id = account_id
self._balance = balance self._history
= []

@property def
balance(self):
return self._balance
def deposit(self, amount):
if amount <= 0: raise InvalidAmountError("Amount must
be positive.")
self._balance += amount

def withdraw(self, amount):
if amount <= 0: raise InvalidAmountError("Amount must
be positive.")
if amount > self._balance: raise
InsufficientBalanceError(f"Insufficient
balance in {self.account_id}"

```
            )
        self._balance -= amount class

Transaction:

    def init (self, operation, account_from=None, account_to=None,
                 amount=0):
        self.operation = operation self.account_from
        = account_from self.account_to =
        account_to self.amount = amountdef str
        (self): return
        (
            f"{self.operation}: "
            f"{self.account_from} -> {self.account_to} " f"{self.amount}"
        )

class Bank:
    def init (self):
        self.accounts = {}
        self.history = []
    def add_account(self, account_id, balance):
```

Riyancee Rana (12402080601119)




```
if balance <
        0: raise
            InvalidAmountError(
                "Initial balance cannot be negative."
            )
        self.accounts[account_id] = Account(
            account_id,
            balance
        )

    def get_account(self, account_id):
        if account_id not in self.accounts:
            raise AccountNotFoundError(
                f"Account {account_id} does not exist."
            )

        return self.accounts[account_id]
    def deposit(self, account_id, amount):
```

```
        account = self.get_account(account_id)
        account.deposit(amount)
        transaction = Transaction(
            "DEPOSIT",
            account_to=account_id,
            amount=amount
        )

        self.history.append(transaction)

    def withdraw(self, account_id, amount):
        account = self.get_account(account_id)

        account.withdraw(amount)
        transaction = Transaction(
            "WITHDRAW",
            account_from=account_id,
            amount=amount
        )

        self.history.append(transaction)

    def transfer(self, from_id, to_id, amount):
        source = self.get_account(from_id)
```

Riyancee Rana (12402080601119)

  destination

```
        = self.get_account(to_id)
        source.withdraw(amount)
        destination.deposit(amount)

        transaction = Transaction(
            "TRANSFER",
            account_from=from_id,
            account_to=to_id,
            amount=amount
        )
        self.history.append(transaction)
    def snapshot(self): return
        {
            account_id: account.balance
            for account_id, account in self.accounts.items()
        } def restore(self,
```

```
    snapshot):
        for account_id, balance in snapshot.items(): self.accounts[account_id]._balance
            = balance

def main(): bank
    = Bank()
    n = int(input().strip()) for
    _ in range(n):
        account_id, balance = input().split()
        bank.add_account
    ( account_id,
    int(balance)) q =
    int(input().strip())

    batch_number = 0
    batch_snapshot =
    None failed_batches =
    []

    for _ in range(q): parts
        = input().split()

        if not parts:
```

Riyancee Rana (12402080601119)



```
continue
            operation = parts[0] if

        operation == "BATCH_BEGIN":

        batch_number += 1

            batch_snapshot = bank.snapshot()
            continue

        if operation == "BATCH_END":
            batch_snapshot = None
            continue

        try:
```

```
        if operation == "DEPOSIT":
            account_id = parts[1] amount =
            int(parts[2])
            bank.deposit(account_id,
            amount)

        elif operation == "WITHDRAW":
            account_id = parts[1] amount =
        int(parts[2])
        bank.withdraw(account_id, amount)
        elif operation == "TRANSFER":
        from_id = parts[1]


           to_id = parts[2]amount = int(parts[3]) bank.transfer(
               from_id
               , to_id, amount
            ) else:
            raise BankError("Unknown operation.")
    except BankError:
        if batch_snapshot is not None:
            bank.restore(batch_snapshot)
            failed_batches.append(batch_number)

            while True: remaining = input().split() if remaining
                and remaining[0] == "BATCH_END":
                    break
```

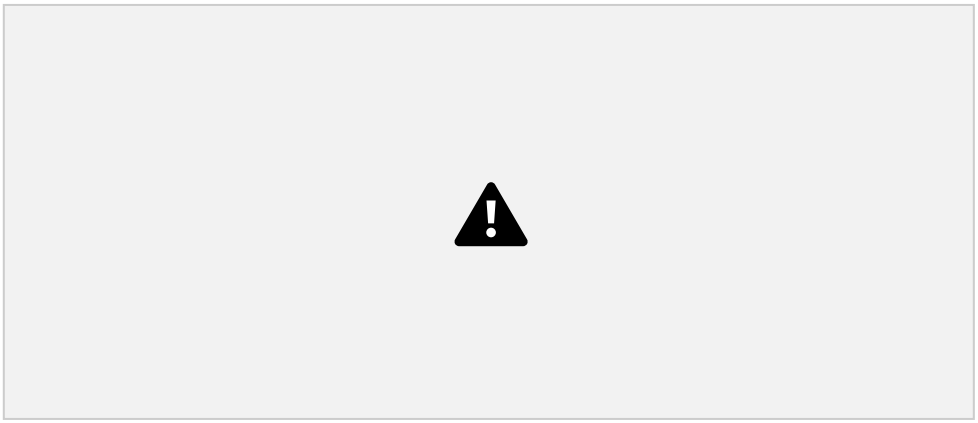
Riyancee Rana (12402080601119)



```
            batch_snapshot = None
    for number in failed_batches:
        print(f"FAILED {number}")
    for account_id in sorted(bank.accounts):
        balance =
        bank.accounts[account_id].balance
        print(account_id, balance)

if name == " main ": main()
```

OUTPUT:



PROBLEM STATEMENT 6: Given a set of Python module names and import relationships, determine a valid loading order. If a circular dependency exists, print CYCLE and the modules involved in one cycle. Ignore duplicate import edges. If multiple modules can be loaded, choose lexicographically smallest module first.

CODE :

```
import heapq def
main():
    n, e = map(int, input().split())
    modules = []

    for _ in range(n): modules.append(input().strip())

    adjacency = { module: [] for module
        in modules
    }

    indegree = { module: 0 for module
        in modules
```

Riyancee Rana (12402080601119)




```
}
    seen_edges =

    set() for _ in

    range(e):

        module_a, module_b = input().split() edge

        = (module_b, module_a)
```

```
    if edge in seen_edges: continue

    seen_edges.add(edge)

    adjacency[module_b].append(module_a
) indegree[module_a] += 1 heap = []

for module in modules:
    if indegree[module] == 0:
        heapq.heappush(heap, module)order = []
        while heap:

    current =

    heapq.heappop(heap)

    order.append(current)

    for dependent in adjacency[current]: indegree[dependent]
        -= 1

        if indegree[dependent] == 0:
            heapq.heappush(heap,

dependent) if len(order) != n:

print("CYCLE")

    state = {module: 0 for module in modules}
    parent = {} cycle = [] def dfs(node):
    state[node] = 1 for next_node in
    adjacency[node]:
```

Riyancee Rana (12402080601119)



```
if
        state[next_node] == 0: parent[next_node]

            = node

            if
                dfs(next_node)
                : return True
```

```
            elif state[next_node] == 1:

                cycle_start = next_node
                cycle.append(cycle_start)
                current = node while current !=
                cycle_start:
                cycle.append(current) current
                = parent[current]
                cycle.append(cycle_start)
                cycle.reverse() return True

        state[node] = 2
        return False

    for module in modules:
        if state[module] == 0:

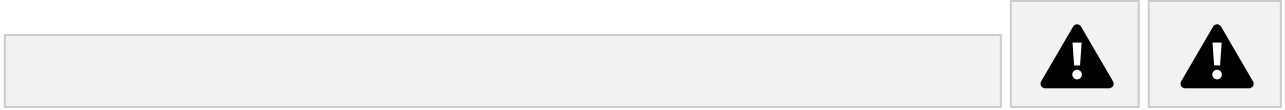
            if dfs(module):
                break
    print("
".join(cycle)) else:
    print(" ".join(order))

if name == " main ": main()
```

OUTPUT:



Riyancee Rana (12402080601119)



PROBLEM STATEMENT 7: Build an interactive calculator that accepts formulas of the form operand operator operand. Operands may be integers, decimals or previously stored variables. Supported operators are +, -, *, / and %. Invalid format, unknown variables, division by zero and unsupported operators must raise separate custom exceptions. The calculator must continue until the user enters quit.

CODE :

```
class
    InvalidFormatError(Exception):
    pass
class UnknownVariableError(Exception):
    pass
class DivisionByZeroError(Exception):
    pass
class
    UnsupportedOperatorError(Exception):
    pass
def get_operand(value, variables):
    try:
        return float(value)
    except ValueError: pass

    if value in variables:
        return
        variables[value]

    raise UnknownVariableError( f"Unknown
        variable: {value}"
    )

def calculate(left, operator, right):
    if operator == "+":
        return left + right

    if operator == "-":
        return left - right

    if operator == "*": return
        left * right

    if operator == "/":
        if right == 0:
```

Riyancee Rana (12402080601119)



```
raise
            DivisionByZeroError( "Cannot
                divide by zero."
            )
```

```
        return left / right if

    operator == "%":

        if right == 0:
            raise DivisionByZeroError(
                "Cannot use modulo with zero."
            )
        return left % right

    raise UnsupportedOperatorError(
        f"Unsupported operator: {operator}"
    )

def format_result(value):
    if value.is_integer():
        return

str(int(value)) return

str(value) def main():

variables = {}

    print("Interactive Calculator") print("Enter
    'quit' to exit.")

    while True:

        try:

            line = input().strip()

            if line.lower() == "quit":
                break

            if not line:
```

Riyancee Rana (12402080601119)



```
raise
                InvalidFormatError( "Empty
                    input."
```

```
    )

if "=" in line: parts =
    line.split("=") if
    len(parts) != 2:
        raise InvalidFormatError(
            "Invalid assignment format."
        )
    variable = parts[0].strip()
    value_text = parts[1].strip()

    if not variable.isidentifier():
        raise InvalidFormatError(
            "Invalid variable name."
        )
        if not value_text:
        raise
            InvalidFormatError(
            "Missing value."
        )

    value = get_operand(
        value_text,
        variables
    )
    variables[variable] = value
    continue
parts = line.split() if
len(parts) != 3: raise
InvalidFormatError(
        "Formula must be: operand operator operand"
    )
left_text, operator, right_text = parts

left = get_operand(
    left_text,
    variables
)
```

Riyancee Rana (12402080601119)



```
right =
    get_operand(
```

```
            right_text,variables
            )

            result =
                calculate( left,
                operator,
                right
            )

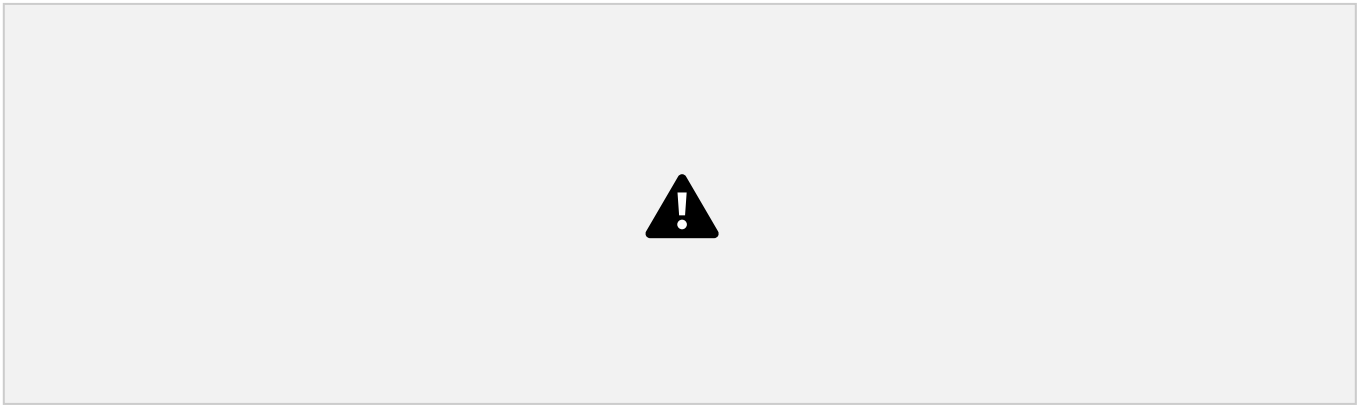
            print(format_result(result))

        except DivisionByZeroError as error:
            print("DivisionByZeroError")
            print(error)
except UnsupportedOperatorError:
            print("UnsupportedOperatorError")

 except UnknownVariableError as error: print("UnknownVariableError")
            print(error)

except InvalidFormatError as
            error:
            print("InvalidFormatError"
            ) print(error)
if name == " main ": main()
```

OUTPUT:



PROBLEM STATEMENT 8: A server produces several text log files. Create a program that scans all files in a folder, builds an inverted index mapping each normalized token to the list of file names and line numbers where it appears, stores the index using pickle, and compresses both original logs

and index into a zip file. The program must also support a search mode that loads the pickle index and answers token queries.

CODE :

```
import os
import re
import pickle
import
zipfile
TOKEN_PATTERN = re.compile(r"[A-Za-z0-9_]+")


def tokenize(line):


    return [
        token.lower()
        for token in TOKEN_PATTERN.findall(line)
    ]
def build_index(folder_path, zip_name):
    if not os.path.isdir(folder_path):
        print("Invalid folder path.")
        return
    index = {}

    total_files = 0 total_lines = 0 for
    filename in os.listdir(folder_path):
        full_path = os.path.join(
            folder_path, filename
        )

        if not

        os.path.isfile(full_path):

        continue total_files += 1 try:

            with open(
                full_path,
                "r",
                encoding="utf-8",
                errors="ignore"
            ) as file:
```

Riyancee Rana (12402080601119)



for

```
            line_number, line in
                enumerate( file, start=1
            ):

                total_lines += 1

                tokens = tokenize(line) for
                token in set(tokens):

                    if token not in index:
                        index[token] = []

                    index[token].append(
                        (filename, line_number)
                    )

        except OSError:
            continue

pickle_path = os.path.join(
    folder_path,
    "index.pkl"
)

with open(pickle_path, "wb") as file:
    pickle.dump(index, file)

with zipfile.ZipFile(
    zip_name, "w",
    zipfile.ZIP_DEFLATED
) as archive:
    for filename in os.listdir(folder_path):
        full_path = os.path.join(
            folder_path,
            filename
        )

        if os.path.isfile(full_path):
```

```
            if os.path.abspath(full_path) == os.path.abspath( zip_name
            ):
```

Riyancee Rana (12402080601119)

  continue

```
            archive.write(
                full_path,
                arcname=filename
            )
    print(f"FILES {total_files}")
    print(f"LINES {total_lines}")
    print(f"TOKENS
     {len(index)}")

def search_index(pickle_path, queries): try:
        with open(pickle_path, "rb") as file: index
           = pickle.load(file)

    except FileNotFoundError:
        print("Pickle index not found.") return

    for query in queries: token =
        query.lower() matches =
        index.get(token, [])
        print(f"{token}:")

        for filename, line_number in matches:
            print(f"{filename}:{line_number}")

def main(): mode =
    input().strip().upper() if
    mode == "BUILD":
        parts = input().split() if


        len(parts) != 2:


        print("Invalid BUILD


        format.")return
```

```
        folder_path = parts[0] zip_name
        = parts[1]
```

Riyancee Rana (12402080601119)



```
        build_index(
            folder_path,
            zip_name
        )
    elif mode == "SEARCH": parts
        = input().split()

        if len(parts) < 2:
            print("Invalid SEARCH format.")
            return
        pickle_path = parts[0]
        q = int(parts[1])
        queries = []

        for _ in range(q): queries.append(input().strip())

        search_index(
            pickle_path,
            queries
        ) else:
        print("Invalid mode. Use BUILD or
SEARCH.") if name == " main " : _ _
main()
```

OUTPUT:



PROBLEM STATEMENT 9: Simulate a job scheduler where each job has job_id, priority, duration and required resource count. Multiple worker threads execute jobs. Higher priority jobs must be processed first; jobs with same priority use earlier arrival order. The program must maintain thread-safe job assignment and produce a final execution report.

CODE :

```
import heapq import
threading
from dataclasses import dataclass
@dataclass
class Job:
```

Riyancee Rana (12402080601119)




```
arrival: int
    job_id: str
    priority: int
    duration: int
    resources:
    int order: int
def simulate(jobs, worker_count):

   jobs.sort(
       key=lambda job: (
       job.arrival,
       job.order
       )
   )
    waiting = []
    workers = [
    (0, i + 1)
       for i in range(worker_count)
   ]
   heapq.heapify(workers)
   results = [] index = 0 current_time
   = 0 while index < len(jobs) or
   waiting:

       if not waiting and index < len(jobs):
           current_time = max( current_time,
              jobs[index].arrival
           )

       while (
           index < len(jobs)and jobs[index].arrival <= current_time
       ):
          job = jobs[index]
           heapq.heappush(
```

```
            waiting,
            (
                -job.priority,
                job.arrival,
                job.order,
                job
            )
        )
```

Riyancee Rana (12402080601119)




```
index += 1
    finish_time, worker_id = heapq.heappop(
        workers
    )
    current_time = max(
        current_time,
        finish_time
    )
    while (
        index < len(jobs)
        and jobs[index].arrival <= current_time
    ):
        job = jobs[index]
        heapq.heappush(
            waiting,
            (
                -job.priority,
                job.arrival,
                job.order,
                job
            )
        )

        index += 1
    if not waiting: heapq.heappush(
            workers,
            (current_time, worker_id)
        )
        continue
    _, _, _, job = heapq.heappop(waiting)

    start_time = max(
```

```
            current_time,
            job.arrival,
            finish_time
        )

        finish_time =
            ( start_time +
            job.duration
        )
```

Riyancee Rana (12402080601119)




```
        waiting_time =
            ( start_time -
            job.arrival
        )
        results.append(
            (
                job.job_id,
                worker_id,
                start_time,
                finish_time,
                waiting_time
            )
        )

        heapq.heappush(
            workers,
            (
                finish_time,
                worker_id
            )
        )
        current_time = start_time
    return results
def main():
    w, n = map(int, input().split()) if
    not (1 <= w <= 64):
        raise ValueError("Workers must be between 1 and 64.")
        jobs = []
    for order in range(n):
        arrival, job_id, priority, duration, resources = (input().split()
```

```
        )
        job = Job( int(arrival),
            job_id,
            int(priority),
            int(duration),
            int(resources),
            order
        )
        jobs.append(job)
```

Riyancee Rana (12402080601119)



results =

```
    simulate(
        jobs, w
    )
    total_wait = sum(
        result[4] for result
        in results
    )
    for job_id, worker_id, start, finish, wait in results: print(
            f"{job_id} W{worker_id} "
            f"{start} {finish}"
      )
    average_wait = (
        total_wait / len(results)
        if results
        else 0
    )
    print(
        f"AVG_WAIT {average_wait:.2f}"
    )
if name == " main ": main()
```

OUTPUT:

PROBLEM STATEMENT 10: Create a Tkinter GUI application to manage student assignment submissions. The application must support adding a student, adding a submission, updating marks, filtering by pending/completed status, and exporting a CSV report. Data must persist locally using JSON or CSV files. Input fields must be validated before saving.

CODE :

```
import tkinter as tk
```

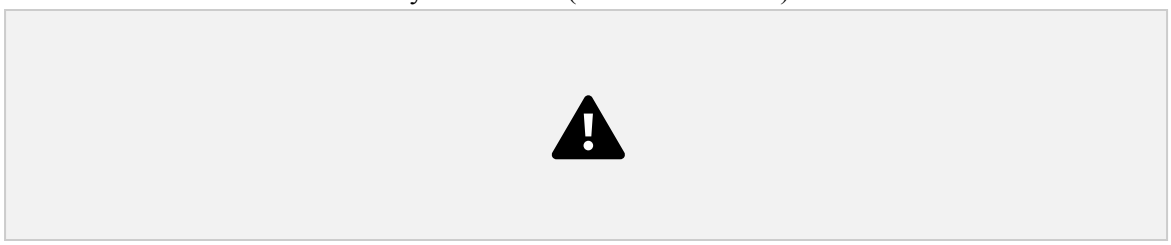
```
from tkinter import ttk, messagebox, filedialog
import json import csv import os

DATA_FILE = "assignments.json" class

AssignmentTracker:

    def init (self, root):
```

Riyancee Rana (12402080601119)






```
self.root =
        root
        self.root.title("Assignment
        Tracker")
        self.root.geometry("900x600")

        self.records = self.load_data()
        self.create_widgets()

        self.refresh_table()

    def load_data(self):

        if not os.path.exists(DATA_FILE):
            return []
try:

            with open(
                DATA_FILE,
                "r",
                encoding="utf-8"
            ) as file:
return json.load(file)

        except (json.JSONDecodeError, OSError):
            return []
    def save_data(self):
        with open(
```

```
            DATA_FILE,
            "w", encoding="utf
            8"
        ) as file:

            json.dump(
                self.records,
                file,
                indent=4
            )
    def create_widgets(self):
        title = tk.Label(
            self.root, text="Student Assignment
            Tracker",
```

Riyancee Rana (12402080601119)



```
            font=("Arial", 18, "bold")
        )
        title.pack(pady=10)

        input_frame = tk.Frame(self.root)
        input_frame.pack(pady=5)

        tk.Label(
            input_frame,
            text="Enrollment:"
        ).grid(row=0, column=0, padx=5)

        self.enrollment_entry = tk.Entry(
            input_frame
        )

        self.enrollment_entry.grid(
            row=0,
            column=1,
            padx=5
        )

        tk.Label(
            input_frame,
            text="Name:"
        ).grid(row=0, column=2, padx=5)
```

```
        self.name_entry = tk.Entry(
            input_frame
        )

        self.name_entry.grid(
            row=0,
            column=3,
            padx=5
        )

        tk.Label(
            input_frame,
            text="Assignment:
            "
        ).grid(row=1, column=0, padx=5)
```

Riyancee Rana (12402080601119)



```
        self.assignment_entry = tk.Entry(
            input_frame
        )

        self.assignment_entry.grid(
            row=1,
            column=1,
            padx=5
        )

        tk.Label(
            input_frame,
            text="Status:"
        ).grid(row=1, column=2, padx=5)

        self.status_combo = ttk.Combobox(
            input_frame, values=["Pending",
            "Completed"], state="readonly"
        )
        self.status_combo.set("Pending")


        self.status_combo.grid(
            row=1,column=3, padx=5
```

```
)

tk.Label(
    input_frame, text="Marks:"
).grid(row=2, column=0,
padx=5)

self.marks_entry = tk.Entry(
    input_frame
)

self.marks_entry.grid(
    row=2,
    column=1,
    padx=5
)
```

Riyancee Rana (12402080601119)



```
tk.Label(
    input_frame,
    text="Remarks:"
).grid(row=2, column=2, padx=5)

self.remarks_entry = tk.Entry(
    input_frame
)

self.remarks_entry.grid(
    row=2,
    column=3,
    padx=5
)

button_frame = tk.Frame(self.root)
button_frame.pack(pady=10)

tk.Button(
    button_frame, text="Add
    Submission",
    command=self.add_submissio
    n
).grid(row=0, column=0, padx=5)
```

```
tk.Button(
    button_frame, text="Update
    Marks",
    command=self.update_mark
    s
).grid(row=0, column=1, padx=5)

tk.Button(
    button_frame, text="Export
CSV", command=self.export_csv
).grid(row=0, column=2, padx=5)

tk.Button(
    button_frame,
    text="Clear",
    command=self.clear_fields
```

Riyancee Rana (12402080601119)




```
).grid(row=0, column=3, padx=5)

filter_frame = tk.Frame(self.root)
filter_frame.pack(pady=5)

tk.Label(
filter_frame,
text="Filter:"
).pack(side=tk.LEFT)

self.filter_combo = ttk.Combobox(
filter_frame,
values=[
"All",
"Pending",
"Completed"
],
state="readonly"
)
self.filter_combo.set("All")
self.filter_combo.pack(
```

```
side=tk.LEFT,
padx=5
)
```

OUTPUT:

